Reader-Writer Synchronization Using a Client-Server System

Group Members:
Joy “Wilson” Skipper
Ian Smith

May 13, 2026
COSC 519

Project Overview:

The core objective of the project is to demonstrate and solve the Reader-Writer problem through the implementation of a client-server system. The server simulates multiple clients accessing the same server through the utilization of processes and threads. The server allows proper regulation of requests to the server by allowing multiple readers to access the data on the server while restricting any access to the server once a writer has begun modifying the data on the server. Synchronization techniques such as the use of locks and correctness constraints prevent data conflicts from occurring due to a client reading data that is being modified by a writer and ensure consistency of the data being accessed by the readers.

Objectives & motivation:

Several objectives were set for the project. The first was picking a programming language and environment that would allow our team to effectively implement the reader-writer simulation using a web server. After some research and discussion, python was chosen as the language to be used with Flask as the web server. This provides a simple but effective development environment allowing our team to perform the aims of project.

The core objective and motivation are to simulate the reader-writer problem visually on a webpage using the web server. To help make the visualization more interesting, a hotel booking system was used as the example for the simulation. The system uses an SQLite database that stores the values for rooms in the hotel which is the component the readers and writers attempt to access. Threads are created to serve as the readers and writers to allow these multiple readers and writers to act concurrently. The server displays on the page when a reader or writer accesses the database and demonstrates the reader-writer logic on the page. Writer priority was taken for the reader-writer approach for this project as it fits the example demonstrated.

For example, the web page will show how multiple readers can quickly access the data in the database seemingly at the same time, but when a writer thread is created and is executing, the reader responses to the server and page will halt until the writer modification to a hotel room value is finished. The writer’s threads actions will be displayed on the web page itself to show that the change has occurred and new or waiting reader threads will access and output those new changes showing the writer thread’s action.

Project Description:

To view the webpage, you first must set up a python3 environment with pip. Use pip to install the following tools:

```
pip install flask
pip install flask_sqlalchemy
```

```
pip install flask_security
pip install flask_socketio
pip install argon2_cffi
```

Finally, navigate to the flask_blog/ directory and run the following command:

```
flask run --port 8080
```

You should see something similar to the following:

```
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:8080
INFO:werkzeug:Press CTRL+C to quit
```

You may then open an internet browser and navigate to:

```
http://127.0.0.1:8080
```

At this point you should see the homepage. There are three webpages to the server: the homepage, the hotel booking page (which is password protected) and the hotel monitoring page. The hotel booking page simulates a writer, which is why you need to input permissions to have writer privileges. The username and password are provided via the sticky note set on the homepage (username: manager@hotel.com password: ilovehotels). The hotel monitoring page simulates a reader, which is why permissions are not needed. Please note that patience of several seconds is required when loading pages or submitting any input into the server.

Booking Page Description:

Here you may select one of five rooms to book in the hotel, and the length of time you would like to book the room in seconds. If you try to book a room that is already currently booked by either the automated writers or by having previously booked with this page, you will be redirected to the home page with a message in red that says “That room is already booked! Please try again.”

Hotel Monitoring Page Description:

Here you may view the current status of all hotel bookings. Under ‘actions,’ you may ‘test readers and writers.’ This generates 10 readers and 5 writers on the backend to simulate multiple simultaneous database clients. The writers create one booking for each type of room for a random number of seconds between 20 and 40. You may also disconnect. This disconnects the socket, but does not give a message, since it cannot once it’s disconnected.

Underneath actions, there is the connection log. This gives test messages showing that the socket has connected. On the right is the main purpose of the page: hotel bookings. Here, there is a message showing the values of the database that generates every 2 or 3 seconds (this is randomly decided by a sleep() function).

Implementation details:

General:

We implemented these objectives through the metaphor of a webpage that books hotel rooms for a number of seconds specified by the user. This includes both webpages where the user can "book" a hotel for a specified number of seconds (writer), view the current hotel bookings (reader) and several simulated reader and writer clients working on the database in the background to simulate having several clients at once on the web server.

Web Server Structure:

The general framework uses Python and the Web Server Gateway Interface (WSGI) web application framework known as Flask. The database setup used SQLAlchemy, an Object Relational Mapper (ORM) for using SQL databases with Python. For live views and changes to the database, Flasks own module SocketIO was used, which creates a permanent, bi-directional communication over a transmission control protocol (TCP) connection. SocketIO was a core component of this setup which allowed the use of multiple readers and writers while viewing a single webpage instance.

Communication between the webpages, which exist in static/templates/, and the database all rests within app.py. Here, the app is created as an instance of the Flask class and represents the web application itself. It's configured to use SQLite, SQLAlchemy, and handle multithreading. SQLAlchemy is then called on the 'app' instance to create the database connection object, labeled 'db.' This is then called, directly or indirectly, for all connections to the database instance. The hotel database model is then defined with the necessary parameters for hotel booking.

With this all set up, there are many functions which fall into three rough categories:

1.) Typical Python functions which handle any kind of general logic, including the majority of the reader-writer logic.

2.) @socketio.event functions, which use SocketIO to handle live changes to the HTML pages. These are typically triggered using Javascript on either the .HTML webpages or another function in app.py itself, using the function name and socketio.emit(<function_name>). However, there are some standardized SocketIO events, such as connect() or disconnect() as well. Emit() can also be done to call an event on the .html pages within the Javascript, rather than solely to call a function within app.py

3.)@app.route() functions, also known as views. These are called when the app navigates to a particular URL within the application. These paths reflect their corresponding .HTML pages, which are called at the end of these functions using render_template(), with some exceptions such as errors, in which another template may be rendered instead.

Finally, there is a one-time setup of the database, including the user with writer permissions, and all hotel rooms within the Hotel model. Main() is then run, which simply runs the app on port 8080.

Reader-Writer Logic:

The reader-writer logic was written using conditional variables and thread locks from the python Threading library. The approach taken for the reader-writer problem is writer-priority meaning that when a new writer thread enters the system, all active reader threads finish execution but any new reader threads that enter the system while there are any writer threads executing or waiting must wait until all writer threads finish executing. To help improve concurrency of threads and allow readers to return values to other functions for output, ThreadPoolExecutor was used to achieve these aims.

Two functions were created, one for readers and one for writers, with each having an entry section that checks whether they can begin execution. For readers threads, if any writer threads are active or waiting, then the new reader thread must wait until all writer threads have left the system. This will also include new writer threads that enter while the reader threads are waiting. Writer threads must wait for all active readers to finish first and there is no other writer thread already executing. Once a thread is finished, they will signal the appropriate thread to start their execution with notify(). The critical section has the thread access the database to either read booking values of rooms (readers) or book a room in the hotel (writers). The exit section serves to notify that a thread has left the system and to notify other threads depending on system status (i.e. a reader thread notifying a writer thread to begin).

Reader and writer are used in a few locations within the server. When navigating to the Booking Page, a reader thread is created to retrieve the names of all the rooms in the hotel. While in the Booking Page, after selecting a room, entering a value in the textbox, and clicking submit, a writer thread is created to update the value in the database for that room. Another writer thread is created after the allocated booking time has passed to update the value back to 0. In the Hotel Monitoring Page, on every ping sent to the server a reader thread is created to display on the page every room in the hotel and its current booking value.

To help simulated the reader-writer logic live visually, a Test Readers and Writers button was implemented. When the button is clicked, a reader thread is generated to retrieve all names of all the rooms in the hotel. After this, ten reader threads are created and are assigned a random room to retrieve the data for. Writer threads for each room in the hotel are created, with each having a random booking time between 20 and 40 seconds. Another group of writer threads are generated that wait equal to the rooms booking time and then enter the entry section to update the value to 0.

Group Member Distribution:

Wilson’s Role

Primarily worked on the setup of the web server. This included setting up the flask environment using SQLAlchemy to create a database to hold the hotel booking data, setting up communication with SocketIO so that live changes could be made, and setting up all webpages. She set up the booking webpage to create changes to the database and mark the hotels as booked, and the reader page which regularly queries the database for the latest data on all hotel bookings. She also labeled the critical sections of code that interface with the database for easier implementation of the reader-writer logic.

Ian's Role:

Primarily worked on implementation of reader and writer logic and proper usage and synchronization of readers and writers. Set up a Test Readers and Writers button to generate 10 readers and a writer for each room for demonstration of simulated logic. Add usage of readers and writers to relevant functions of the web server with readers threads generated whenever a read operation is sent to the database to retrieve room information and writer threads when a booking is made with an update operation.

Challenges and Problems Met:

When setting up the webpages, there were some challenges with setting up live changes that could be made, rather than requiring HTML form actions. This was crucial to the entire reader/writer thread structure, since the entire purpose was to simulate multiple simultaneous changes to the database. Initially, an attempt was made to use PostgreSQL rather than SQLite to take advantage of its “LISTEN” and “NOTIFY” features, which could have been used to detect changes to the database and then react accordingly in the app.py file. However, PostgreSQL requires the real user’s username and password to connect to the database by default. More investigation was done, and eventually it was decided to stay with SQLite as the relational database engine and instead incorporate SocketIO for live communication with and changes to the database. This was also a more versatile option than simply using “LISTEN” and “NOTIFY,” since it involves a bidirectional web socket, allowing a variety of live changes where Javascript could be used to update the webpages accordingly.

Additionally, when testing the interaction between reader and writer threads, sometimes there would be a deadlock where writers would wait forever and since there was a writer waiting to run, new reader threads also waited. This is due to a natural problem with servers where some requests either fail or stall. So, when a reader thread attempts to but fails to access the database, the system halts. This was fixed with a simple try block around the critical section and a finally block around the exit section to allow failed accesses of the database to still update the number of threads in the system. Another issue was that the python Threading library does not allow

threads to return a value to be assigned to another variable. This was important for the reader threads generated by ping since this was needed to properly display the data. ThreadPoolExecutor was used as the solution since it allows this function.

Results Analysis:

The results show the writer priority reader-writer approach was successfully implemented into the web server. The Hotel Monitoring Page visualizes the logic as shown in figure 1. Using the Test Readers and Writers button, when a writer enters the system and a reader is active, the "waiting writer action" is displayed. After the active reader thread finishes, the writer threads execute one at a time while displaying their actions. Reader threads then continue to retrieve data once all writers have finished and left the system. Figure 1 also demonstrates that the reader and writer threads act in a random order and is not predetermined.

Figure 1: Example of the reader-writer logic working in Hotel Monitoring Page.

Booking a room in the Booking Hotel page also correctly updates the database which can be viewed in the Hotel Monitoring Page. In both cases of writer threads booking a room, after the entered amount of time another writer thread is created to update the room booking time back to 0.

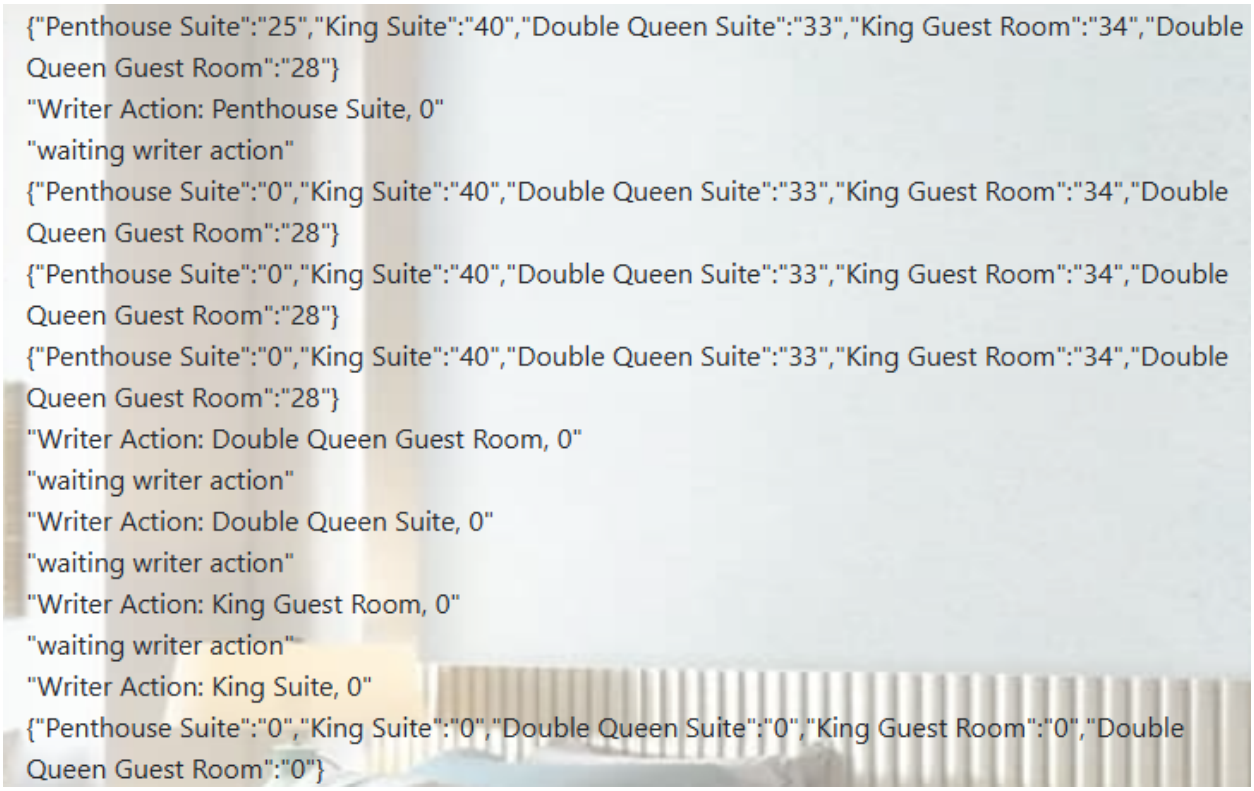

Figure 2. Writer threads updating values of room's booking times back to 0.

Future Work:

Future work on the project could include implementing other approaches to the reader-writer problem such as reader priority or a balance approach. This could demonstrate the difference visually to help improve understanding of the differences between the approaches. Examples could be added to show the how starvation could occur when either reader or writer priority is used.

The aesthetic of the hotel monitoring page could also be improved, such that the live database status feed is viewed as a more user-friendly table that periodically updates its values. A status bar could also be added to show if the table is awaiting writer action for table updates.

References

[1] B. Akash. 2025. Python Flask - Request Object. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/python-flask-request-object/

[2] Michael Bayer et Al. 2026. SQLAlchemy 2.0 Documentation. *Sqlalchemy.org*. Retrieved 2026 from https://docs.sqlalchemy.org/en/20/

[3] Miguel Grinberg. 2014. Easy WebSockets with Flask and Gevent. *blog.miguelgrinberg.com*. Retrieved 2026 from https://blog.miguelgrinberg.com/post/easy-websockets-with-flask-and-gevent?source=post_page-----fb55f9dad100--

[4] Miguel Grinberg. 2018. API Reference. *Flask-Societio.ReadtheDocs*. Retrieved 2026 from https://flask-socketio.readthedocs.io/en/latest/api.html

[5] Khushi Gupta. 2025. Convert Python List to JSON. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/convert-python-list-to-json/

[6] Lito Nicolai. 2021. The basics of role-based access control in SQLAlchemy. *OsoHQ*. Retrieved 2026 from https://www.osohq.com/post/sqlalchemy-role-rbac-basics

[7] Amit Pathak. 2023. Querying and selecting specific column in SQLAlchemy. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/querying-and-selecting-specific-column-in-sqlalchemy/

[8] Amit Pathak. 2024. SQLAlchemy Core - Creating Table. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/sqlalchemy-core-creating-table/

[9] Kushwanth Reddy. 2025. Retrieving HTML Form data using Flask. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/html/retrieving-html-from-data-using-flask/

[10] Kartikeya Shukla. 2026. Introduction to Web development using Flask. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/python-introduction-to-web-development-using-flask/

[11] Sachidanand Thakur. 2025. Creating Instance Objects in Python. *GeeksforGeeks*. Retrieved 2026 from https://www.geeksforgeeks.org/python/creating-instance-objects-in-python/

[12] Ling Thio. 2017. Flask-User v1.0. *Flask User Documentation*. Retrieved 2026 from https://flask-user.readthedocs.io/en/latest/

[13] TutorialsPoint Team. SQLite - Commands. *tutorialspoint*. Retrieved 2026 from https://www.tutorialspoint.com/sqlite/sqlite_commands.htm

[14] Anish Singh Walia. 2020. How To Build a Web Application Using Flask in Python 3. *DigitalOcean*. Retrieved 2026 from https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

[15] Muhammad Waseem. 2024. Flask and Flask-Login: A Guide to Building Secure Web Applications. *Intersystems Developer Community*. Retrieved 2026 from

https://community.intersystems.com/post/flask-and-flask-login-guide-building-secure-web-applications
[16] “Concurrent.futures - launching parallel tasks,” Python documentation, https://docs.python.org/3/library/concurrent.futures.html (accessed May 12, 2026).